

GRUPO I - 1) static String espacoNaN(String palavra)

Objetivo: escolher um **n** aleatório (com $0 < n < \text{palavra.length}()$) e inserir um espaço a cada **n** caracteres, sem terminar com espaço. Exemplo dado: "Banana" com $n=2 \rightarrow$ "Ba na na".

Passos

1. Gerar **n** aleatório **menor** que o comprimento.
2. Percorrer a string e ir juntando ao **StringBuilder**.
3. Depois de escrever **n** **letras**, escrever um espaço **se ainda houver letras**.

```
static String espacoNaN(String palavra) {
    Random rnd = new Random();
    int n = rnd.nextInt(palavra.length() - 1) + 1; // 1..length-1

    StringBuilder sb = new StringBuilder();
    int count = 0;

    for (int i = 0; i < palavra.length(); i++) {
        sb.append(palavra.charAt(i));
        count++;

        // se já escrevemos n chars e ainda faltam chars, põe espaço
        if (count == n && i < palavra.length() - 1) {
            sb.append(" ");
            count = 0;
        }
    }
    return sb.toString();
}
```

3) StoryBuilder / MultiAuthorStory

- **StoryBuilder:** **text** (StringBuilder) e **steps** (começa em 1). **addText** acrescenta e faz **steps++**.
- **MultiAuthorStory:** tem **writers**. O **addText(String s)** chama **addText(s, primeiroAutor)**.

3(a)

i) Atributos e valores

- **text** = "Há muitos, muitos anos, nos sete reinos de Westeros,"
- **steps** = 2

ii) **Ecrã:** **text** = Há muitos, muitos anos, nos sete reinos de Westeros,; **steps** = 2

3(b)

Aqui o tipo real é **MultiAuthorStory**, por isso o **toString()** inclui **writers**.

i) Atributos e valores

- (StoryBuilder) **text** = "As armas e os barões assinalados "
- (StoryBuilder) **steps** = 2
- (MultiAuthorStory) **writers** = ["Orivaldo"] (continua só 1 autor)

ii) **Ecrã:** **text** = As armas e os barões assinalados ; **steps** = 2; **writers**= [Orivaldo]

3(c)

- Começa: **text**="Amor é fogo que", **steps**=1, **writers**=["Camões"]
- **addText(" arde sem se ver", "Clementina")** $\rightarrow$ **steps**=2, **writers**=["Camões","Clementina"]
- **addText(" É ferida que dói")** $\rightarrow$ usa autor inicial ("Camões"), **steps**=3, **writers** não muda.

i) Atributos e valores

- **text** = "Amor é fogo que arde sem se ver É ferida que dói"
- **steps** = 3
- **writers** = ["Camões","Clementina"]

ii) **Ecrã:** **text** = Amor é fogo que arde sem se ver É ferida que dói; **steps** = 3; **writers**= [Camões, Clementina]

GRUPO II (VitiCenter / Fornecedor / Produtor)

II. 1) Classe abstrata **Fornecedor**

Ideia: guardar o nome e os produtos (ex.: num `Map<String, IProduto>`).

O método `calculaPreco(prod, quant)` é **template**: encontra o produto e:

- se `quant == 1` → chama `precoUnidade(p)`
- se `quant > 1` → chama `precoQuantidade(p, quant)`

```
public abstract class Fornecedor implements IFornecedor {
    private String nome;
    private Map<String, IProduto> produtos;

    protected Fornecedor(String nome) {
        this.nome = nome;
        this.produtos = new HashMap<>();
    }

    @Override
    public void novoProduto(IProduto p) {
        produtos.put(p.nome(), p);
    }

    @Override
    public boolean fornece(String nomeProd) {
        return produtos.containsKey(nomeProd);
    }

    @Override
    public double calculaPreco(String prod, int quant) {
        IProduto p = produtos.get(prod);           // assume-se que existe
        if (quant <= 1) return precoUnidade(p);
        return precoQuantidade(p, quant);
    }

    @Override
    public String nome() {
        return nome;
    }

    public abstract double precoUnidade(IProduto p);
    public abstract double precoQuantidade(IProduto p, int q);

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Fornecedor other = (Fornecedor) o;
        return nome.equals(other.nome);
    }

    @Override
    public int hashCode() {
        return nome.hashCode();
    }
}
```

II.2) Classe **Produtor**

- `precoUnidade(p) = p.precoUnitario()`
- `precoQuantidade(p,q) = q * p.precoUnitario() * (1 - desconto)`

```
public class Produtor extends Fornecedor {
    private double desconto; // entre 0 e 1

    public Produtor(String nome, double desconto) {
        super(nome);
        this.desconto = desconto;
    }
}
```

```

    }

    @Override
    public double precoUnidade(IProduto p) {
        return p.precoUnitario();
    }

    @Override
    public double precoQuantidade(IProduto p, int q) {
        return q * p.precoUnitario() * (1.0 - desconto);
    }

    @Override
    public boolean equals(Object o) {
        if (!super.equals(o)) return false;
        Produtor other = (Produtor) o;
        return Double.compare(desconto, other.desconto) == 0;
    }

    @Override
    public int hashCode() {
        return 31 * super.hashCode() + Double.hashCode(desconto);
    }
}

```

GRUPO III

III.1) Regras do compilador (genéricos)

i) `T1<A> varX = new T2<B>();`

➡ Só é aceite se **T1 == T2** e **A == B** (ou seja, os tipos genéricos têm de bater certo). Em geral, **T1<A>** não é compatível com **T1**.

ii) `T1<? extends A> varY = new T2<B>();`

➡ Só é aceite se **T1 == T2** e **B extends A** (B é subtipo de A). O **? extends A** permite guardar um **T1** desde que B seja subtipo de A.